

Building a 10-Foot Experience

React Native for TV

Feb 26, 2026 - Arvin Singla - Confoo



About me

Arvin Singla

- Ontario expat moved to Montreal in 2021
- Lead developer at Prelib
- @arvinsingla most places
- Home automation and 3D printing enthusiast
- mad I couldn't get the domain arv.in



Should you build for TV?

I can't answer that for you...

- Smart TVs and set top boxes are ubiquitous and affordable
- The App market isn't as saturated
- React Native is uniquely positioned to bridge that gap



Key design principles for TV interfaces

Keep it simple

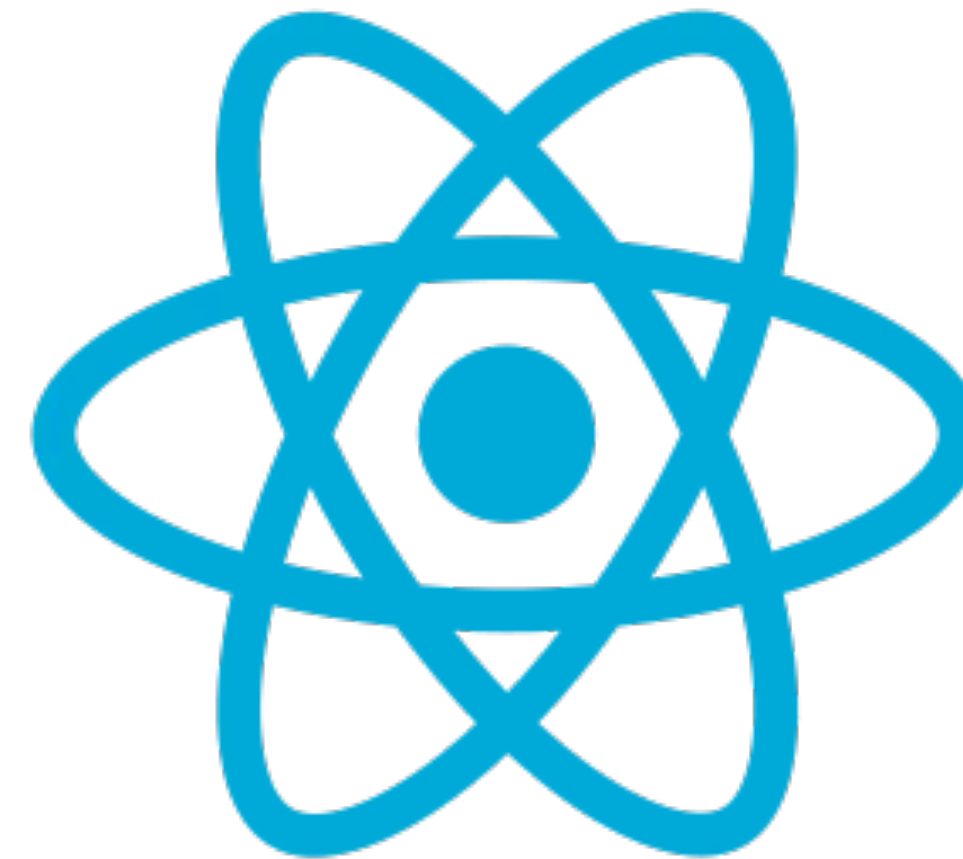
- Ensure clear visuals and readable text at approximately 10 feet
- Prioritize simplicity and low information density
- Avoid overly saturated colors and low contrast between elements
- Start with familiar UI patterns and D-pad navigation



A primer on React

Going strong since 2013

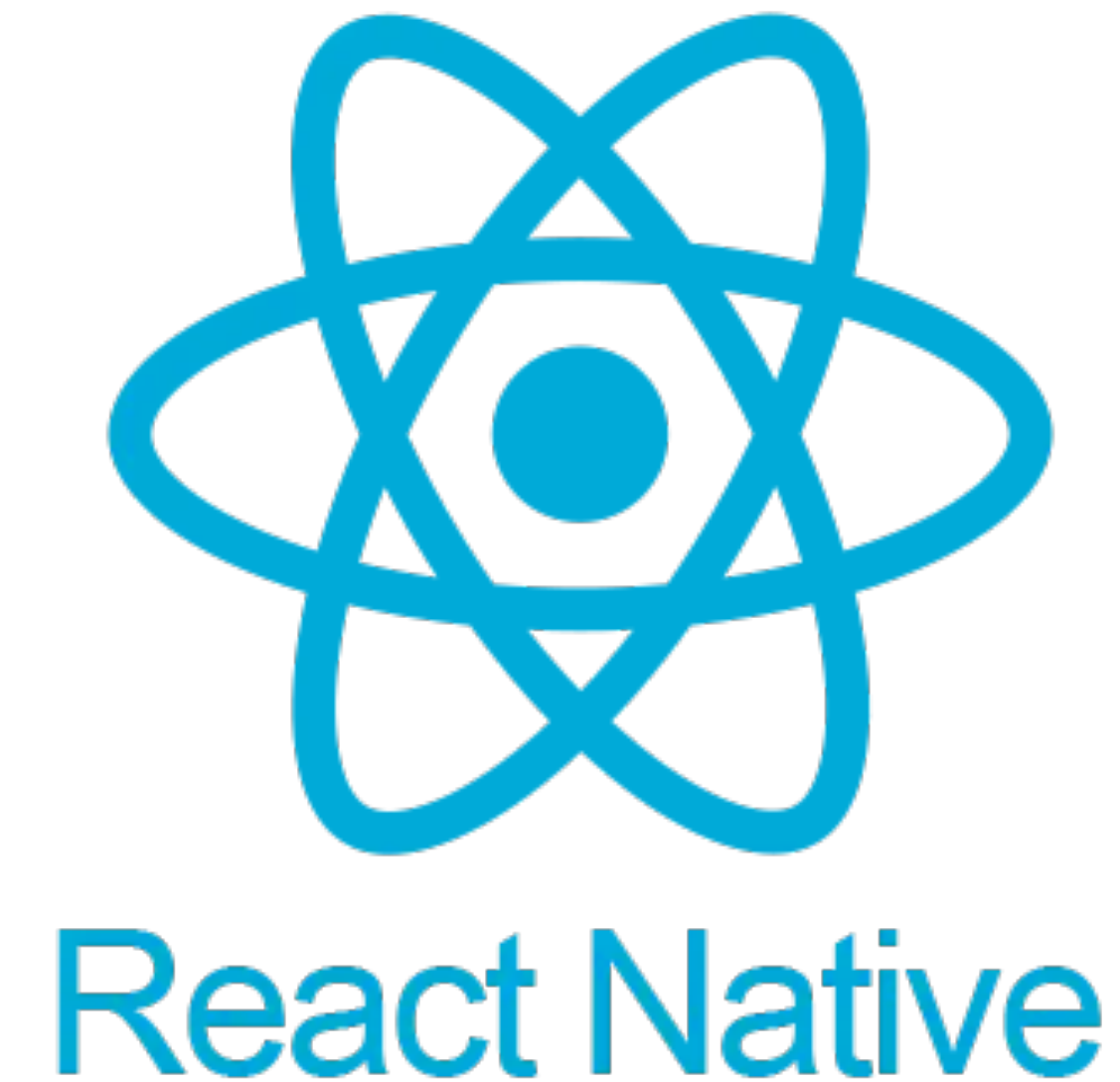
- JS library for building UI developed by Meta
- UI is broken into small, reusable pieces (Component based)
- Each component manages its own state and re-renders when that state changes
- Components are used in HTML like syntax called JSX written directly in JavaScript
- React itself only targets the web



A primer on React Native

Learn once, write anywhere

- Target-agnostic components
`<View>` instead of `<div>`
`<Text>` instead of `<p>`
- Each platform compiles components to real native UI
- JavaScript in its own thread, native UI on the native thread joined by a bridge
- Styles are written in JavaScript using a CSS-like syntax.
- Expo framework is recommended for new projects



JSX example React vs React Native

React

```
function MovieCard({ title, year }) {  
  return (  
    <div className="card">  
      <img src={posterUrl} alt={title} />  
      <h2>{title}</h2>  
      <p>{year}</p>  
    </div>  
  );  
}
```

React Native

```
function MovieCard({ title, year }) {  
  return (  
    <View style={styles.card}>  
      <Image source={{ uri: posterUrl }} />  
      <Text style={styles.title}>{title}</Text>  
      <Text style={styles.year}>{year}</Text>  
    </View>  
  );  
}
```


A primer on Expo

Framework and platform for React Native

- Expo SDK: library of over 100 native modules
- Expo Go: development client for mobile
- Expo CLI: command-line tool for managing app lifecycle
- Expo Router: file based routing
- EAS: cloud services for building, submitting, and updating apps

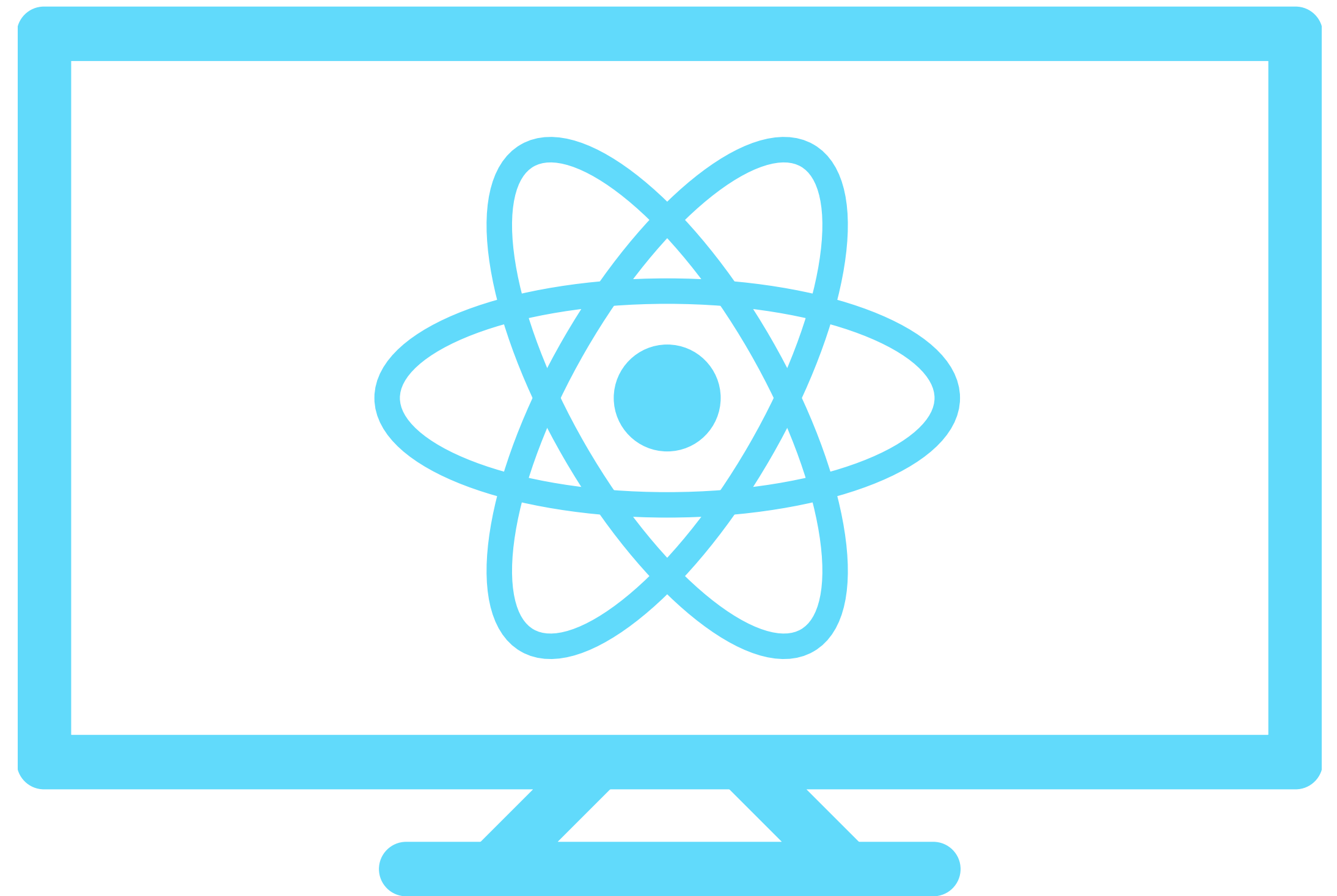


**React Native doesn't support
build targets for TV**

react-native-tvos

It's React Native for tvOS...

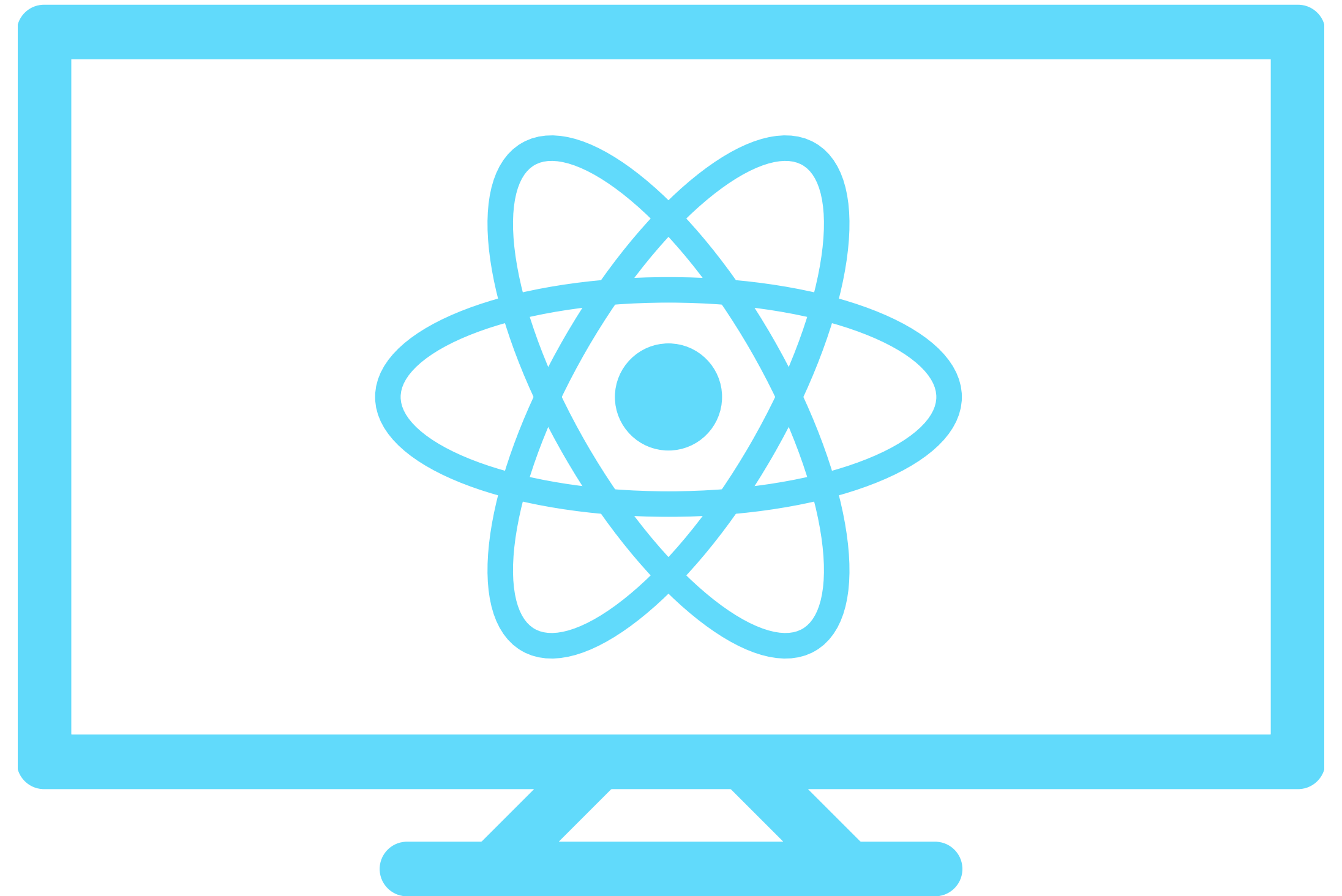
- A fork of React Native that is specially designed with output targets of Apple tvOS and Android TV
- Every release corresponds with a React Native release (latest is 0.84.0)
- Support for Hermes, Fabric, Typescript etc.
- Docs and support are spotty



react-native-tvos

Similarities and differences with React Native

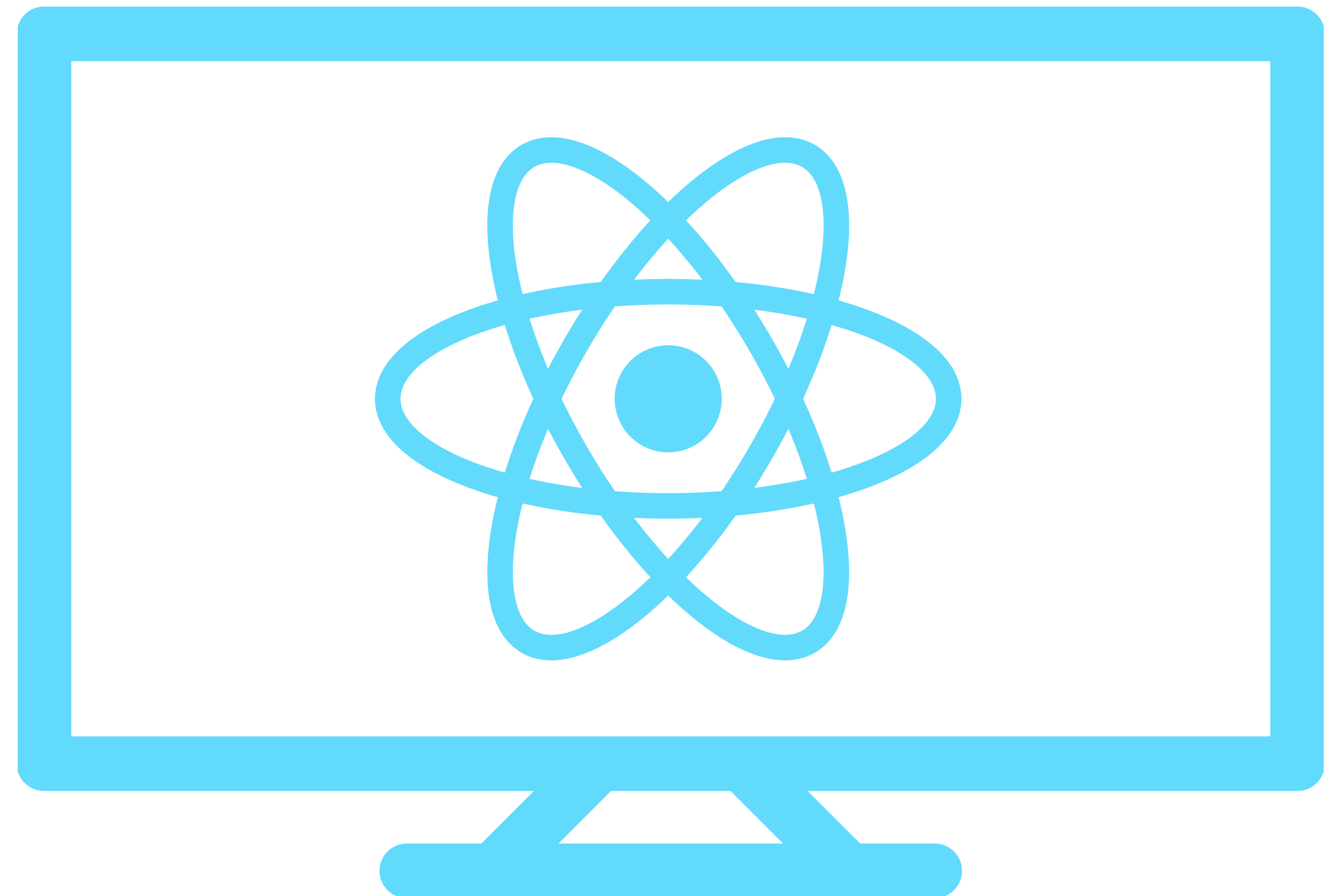
- App structure is identical to core React Native
- No browser or WebView
- Many frameworks, controls, and sensors don't exist
- Input is only accomplished with a remote



react-native-tvos

Input and interaction

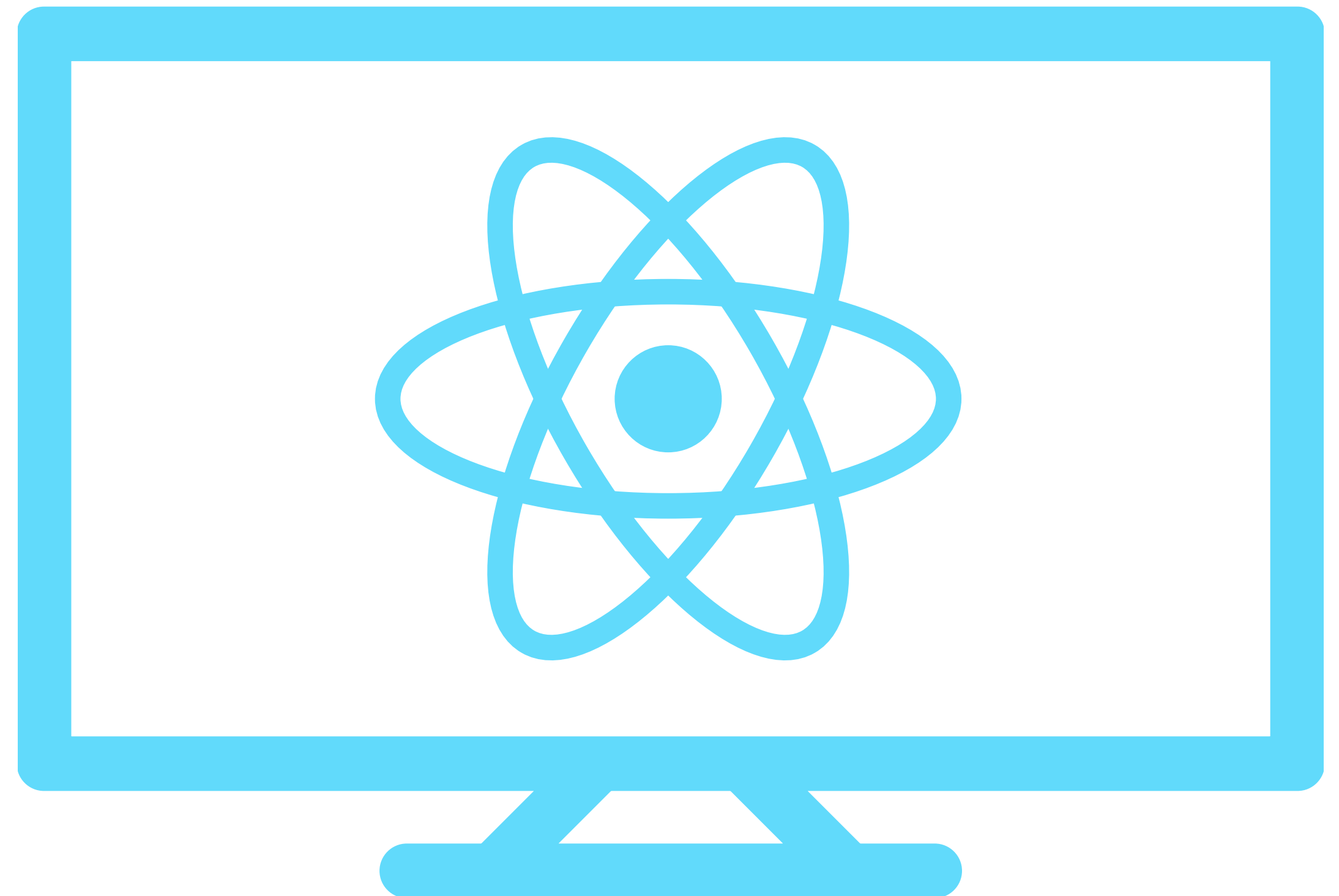
- Remote control input is more like a keyboard than a finger/mouse
- Only 5 inputs that matter: up, down, left, right, select
- No hover states, no swipe gestures, no pinch-to-zoom
- Every interaction must be achievable through focused directional navigation



react-native-tvos

The Focus Engine

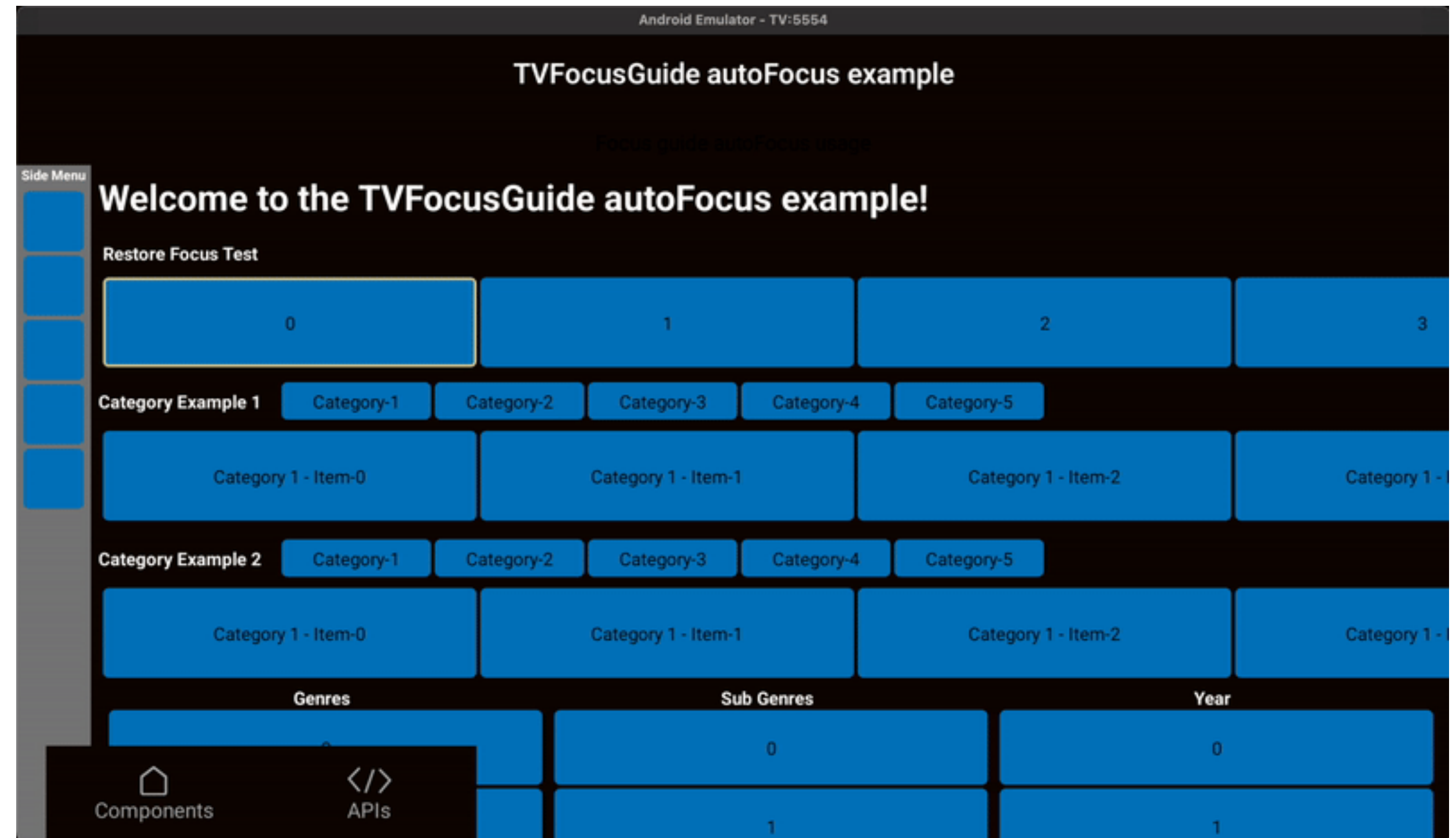
- tvOS has a built-in focus engine that manages which element is active
- "nearest focusable neighbour" in the direction pressed
- Works great when elements are in straight rows/columns
- Breaks down when elements are offset, in grids, or in complex layouts



react-native-tvos

TVFocusGuideView

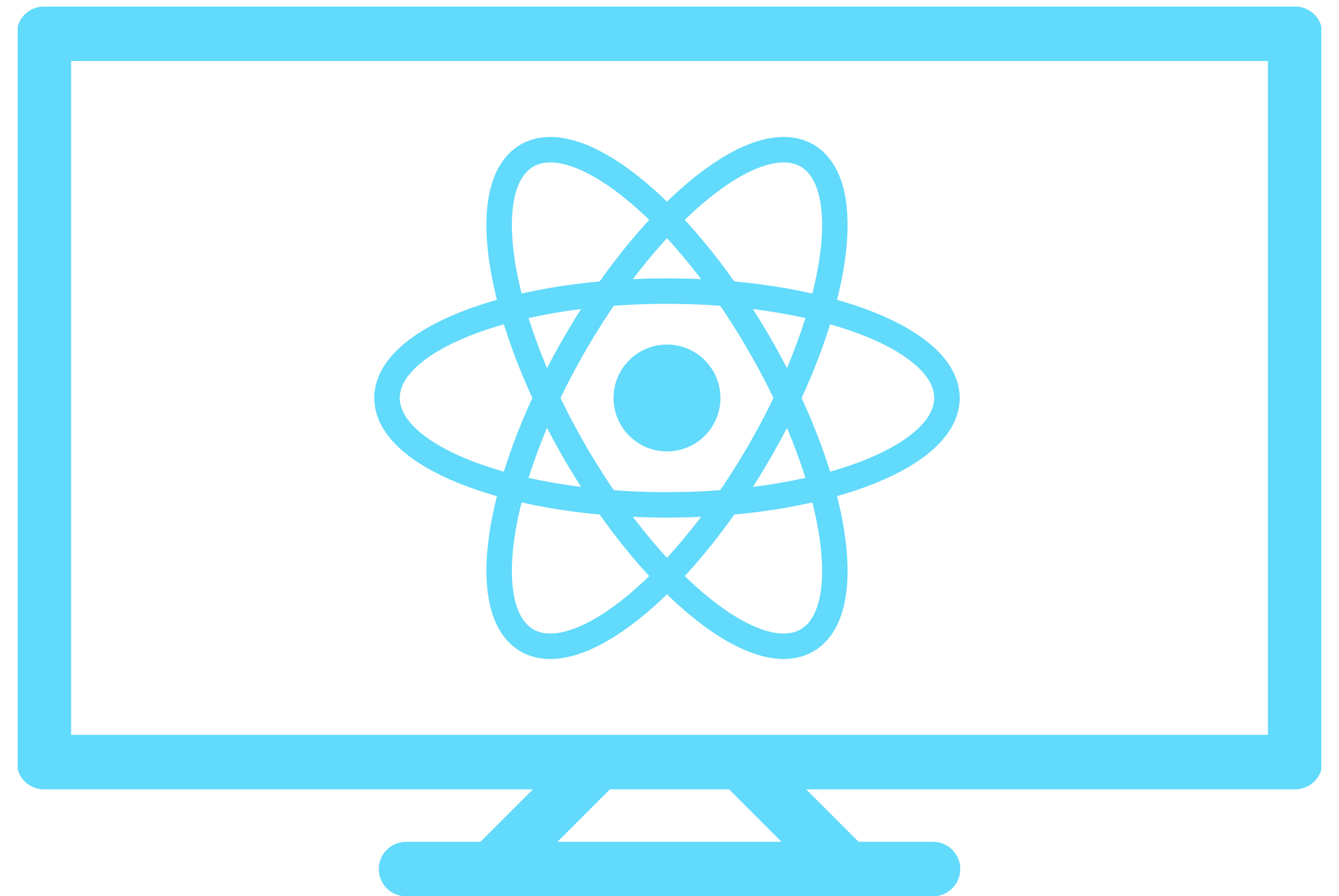
- Ensure that focusable controls can be navigated to even if not directly in line with other controls
- VirtualizeList contents are automatically wrapped with a TVFocusGuideView
- Essential for complex layouts



react-native-tvos

TV-Specific APIs and Components

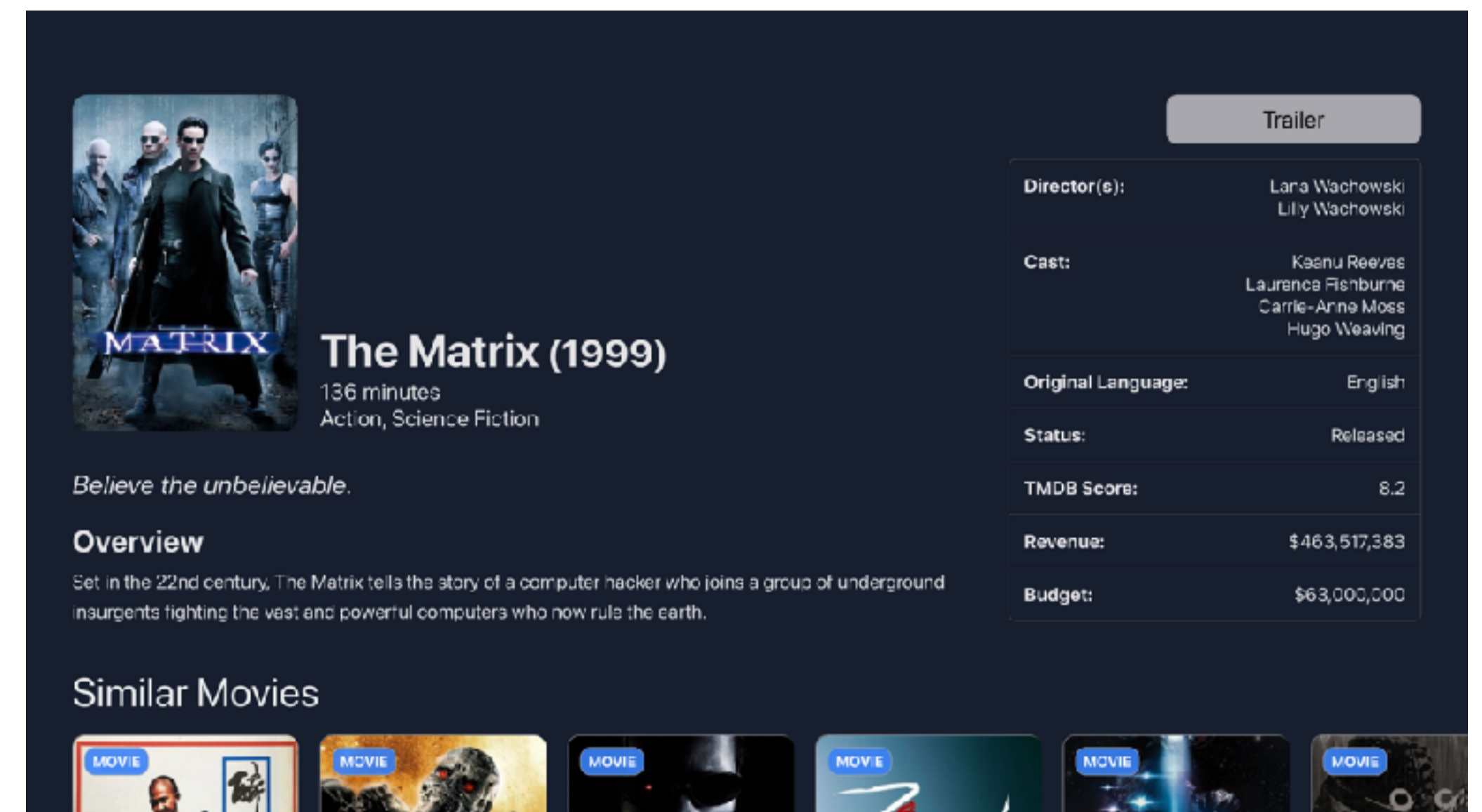
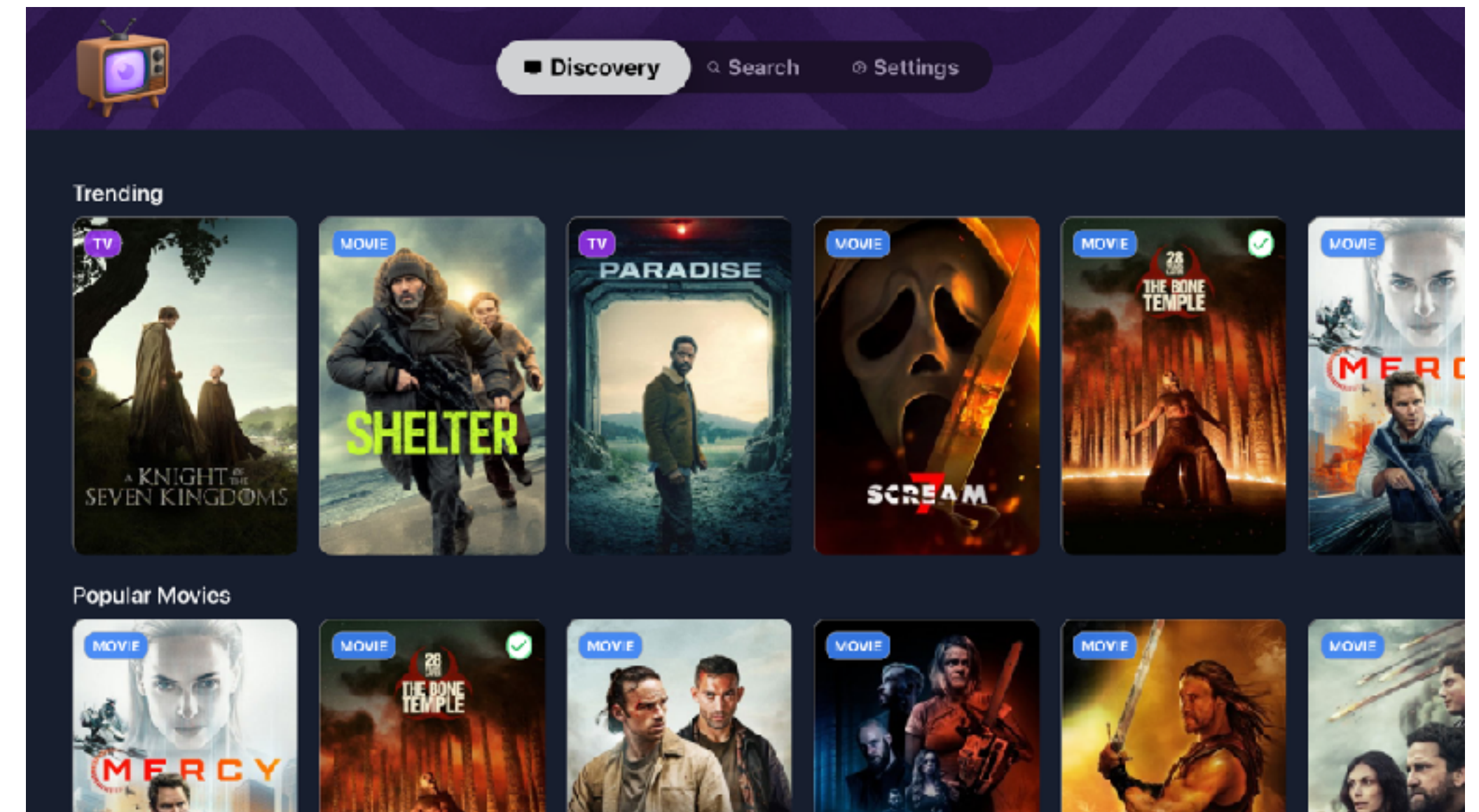
- Platform.isTV and Platform.isTVOS
- TVEventHandler / useTVEventHandler
- TVEventControl and BackHandler
- TVTextScrollView



What did I build?

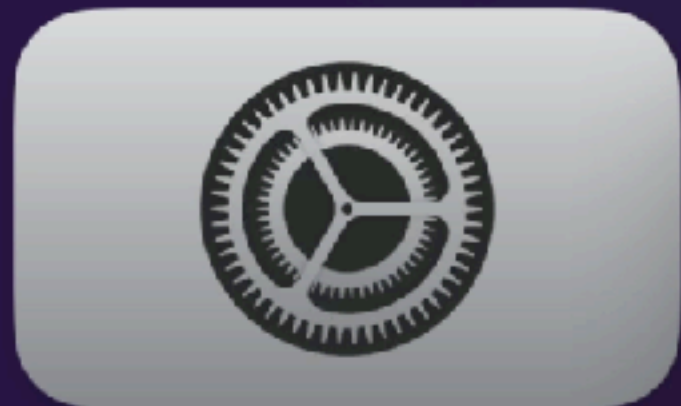
seerr client for AppleTV

- Navigation using expo-router
- API Client generated via seerr OpenAPI spec
- react-query to consume data from seerr API using generated API client
- Zustand for state management
- Custom components for media items, Lists, buttons, toggles, etc





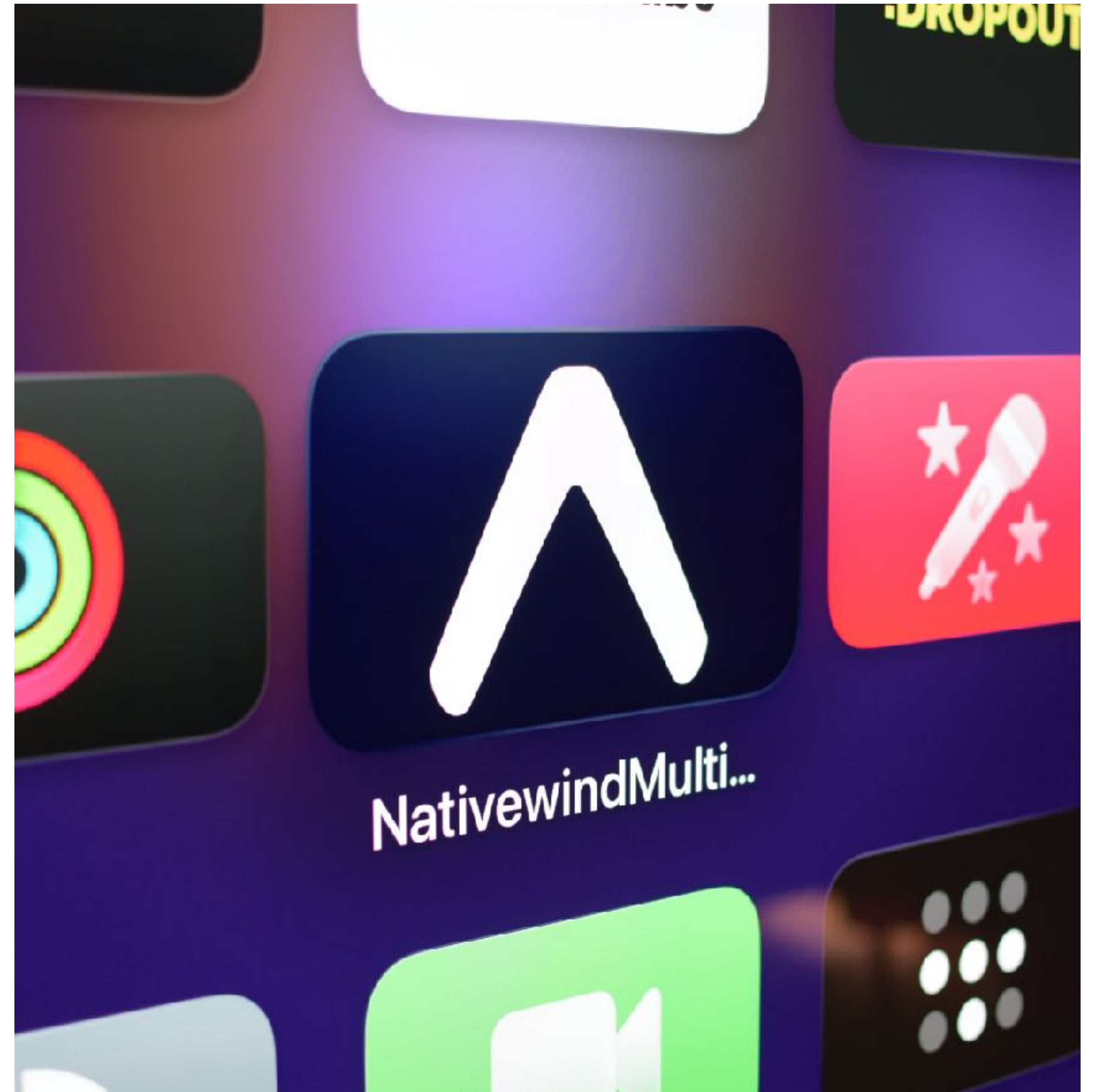
seeTV



Developer Experience

Expo, Debug, Modules

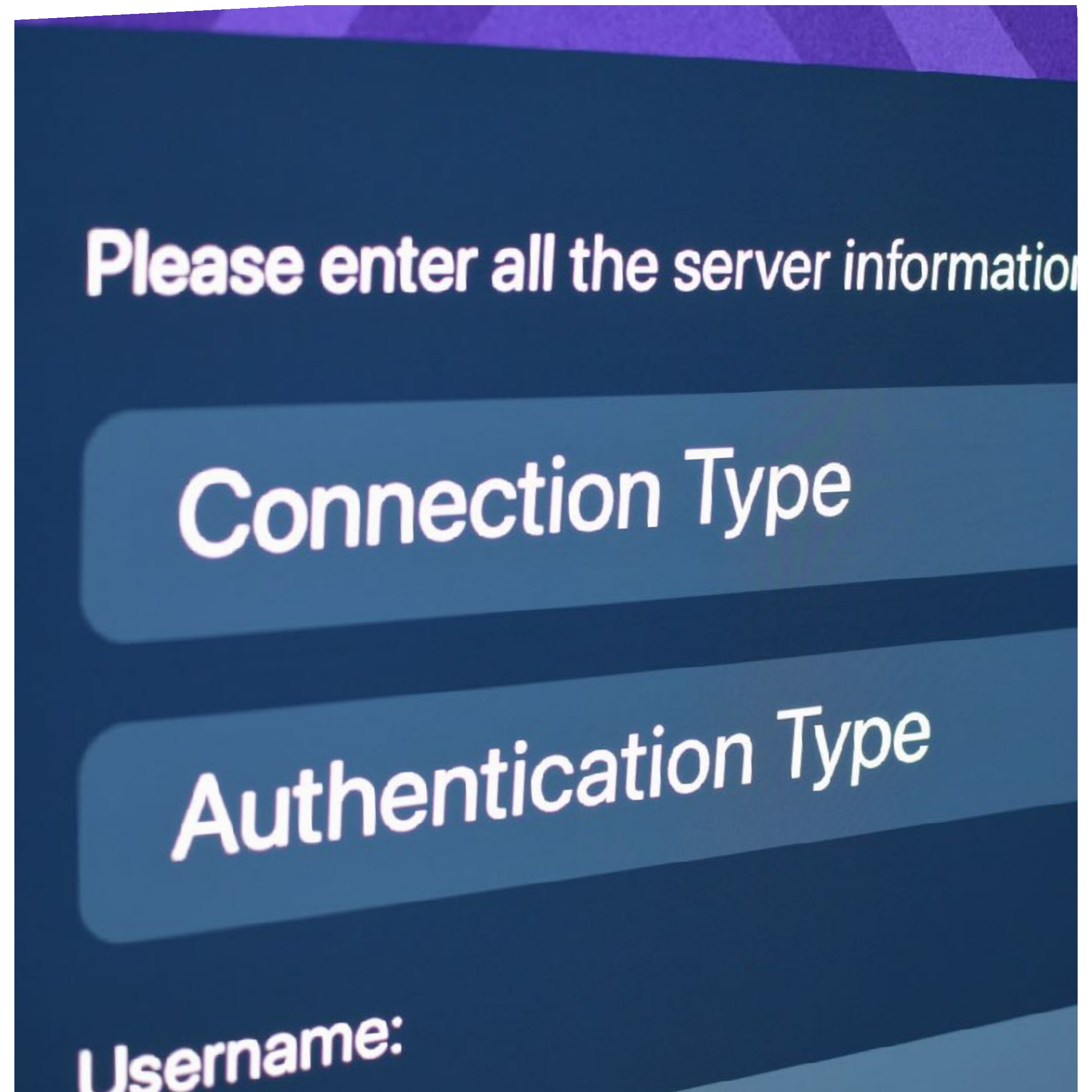
- Many but not all Expo components are available
- HMR and Debug menu supported
- Getting expo client on your device is still undocumented 🤔
- Some 3rd party RN modules are supported.



Developer Experience

Limited native components

- No stable component libraries exist for TV optimized interface elements
- Tailwind is supported via Nativewind
- @expo/ui/swift-ui is promising but still very early beta
- Some native UI elements exist (Alert, NativeTabs)



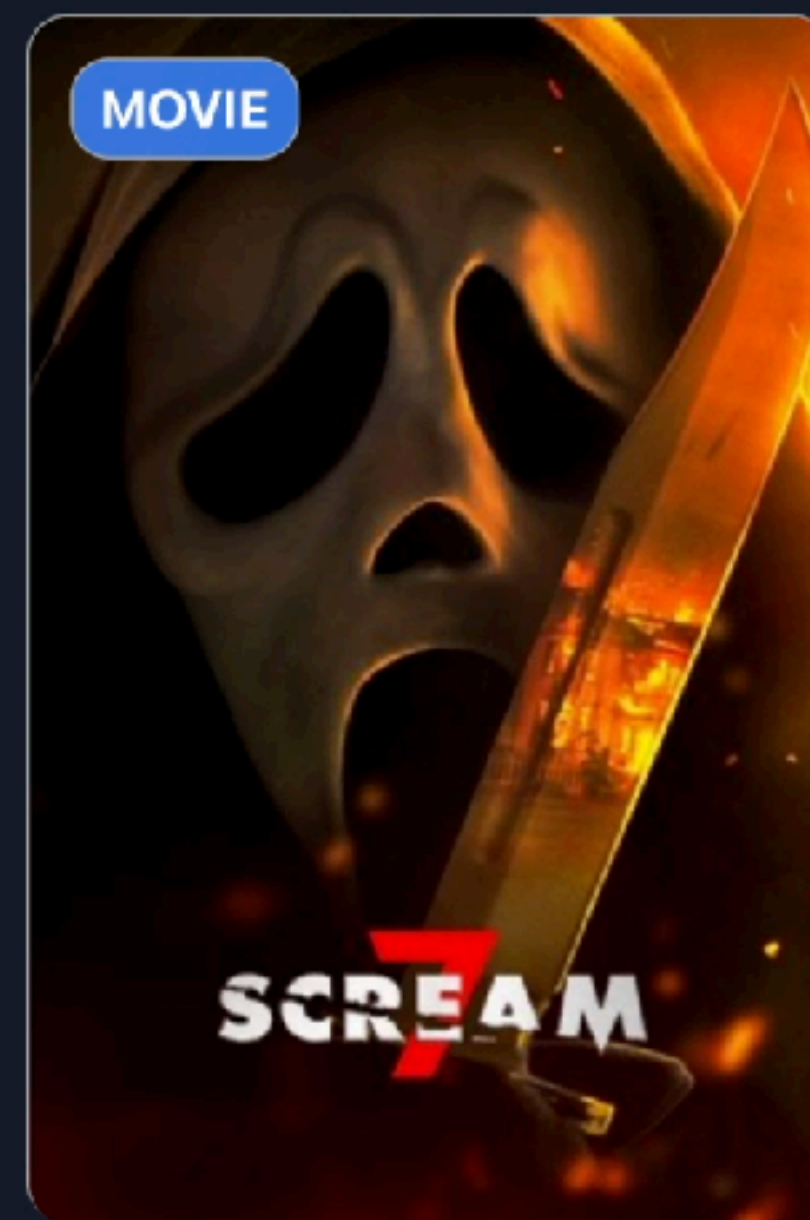


Discovery

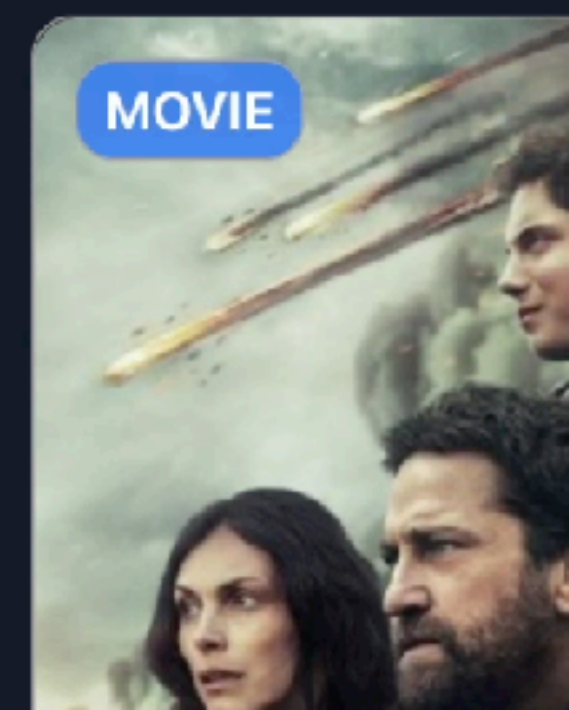
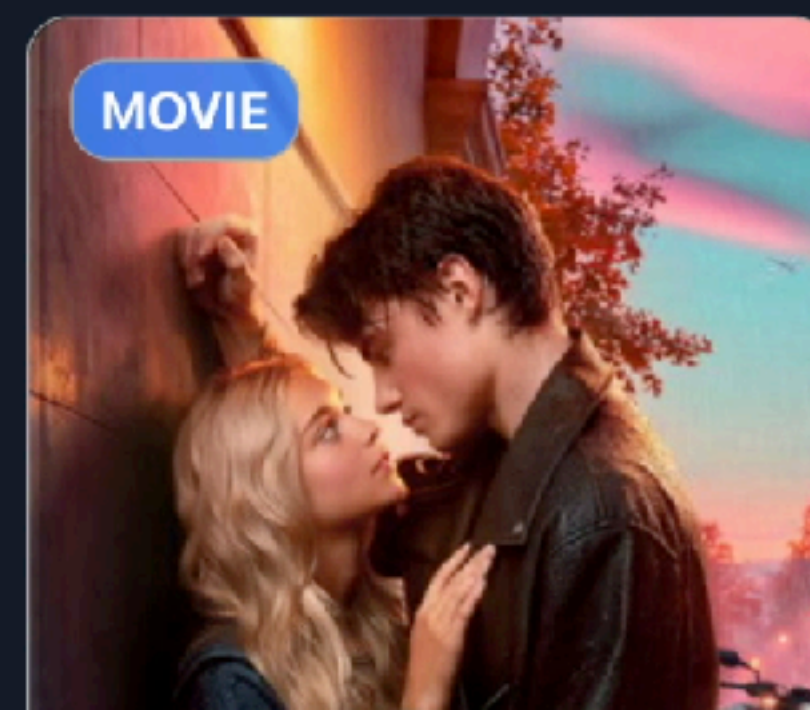
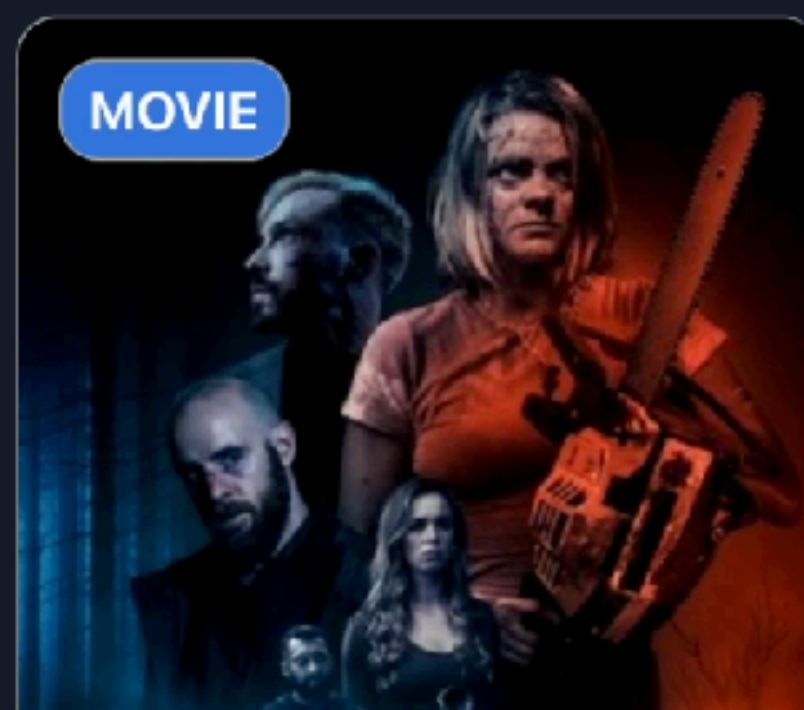
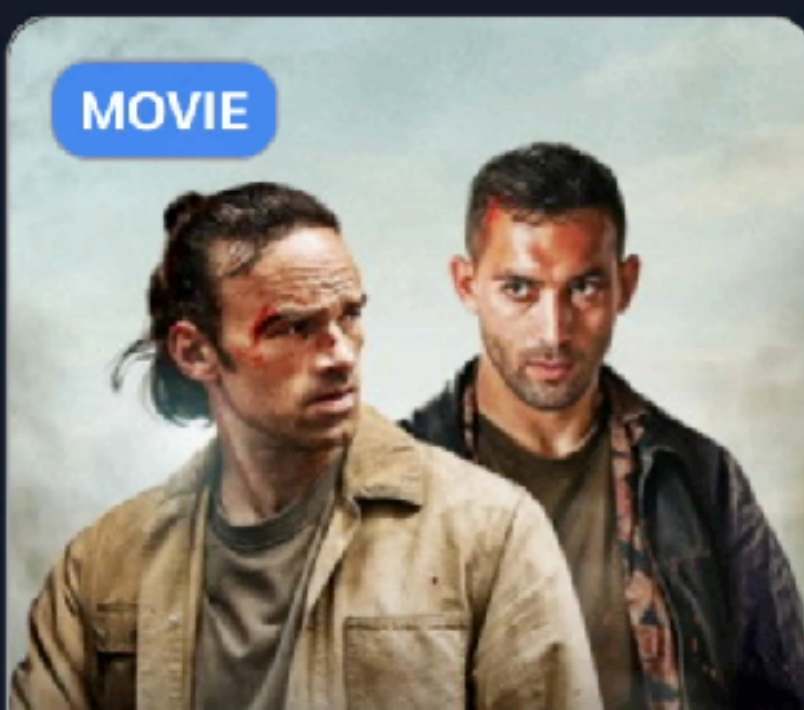
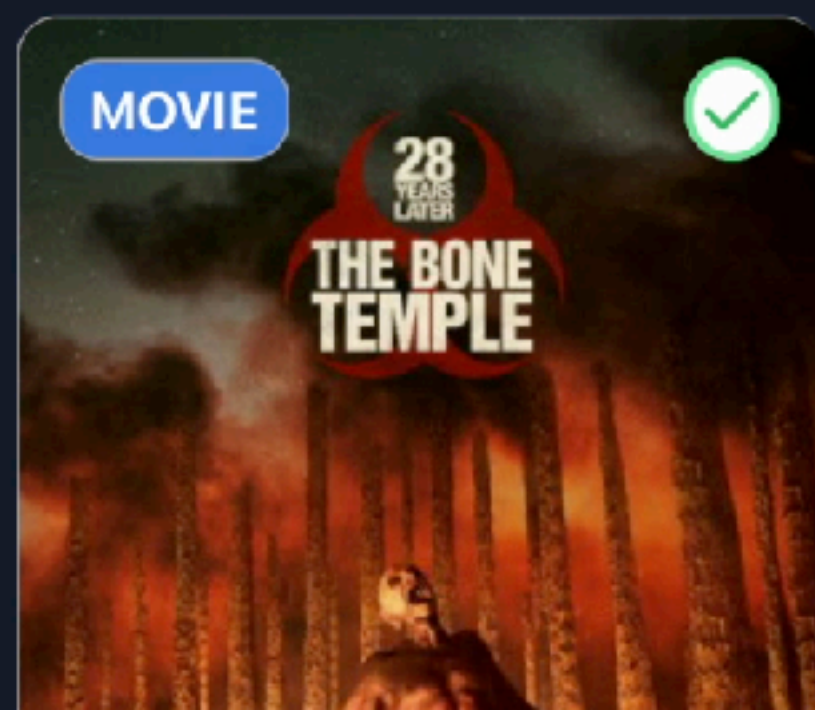
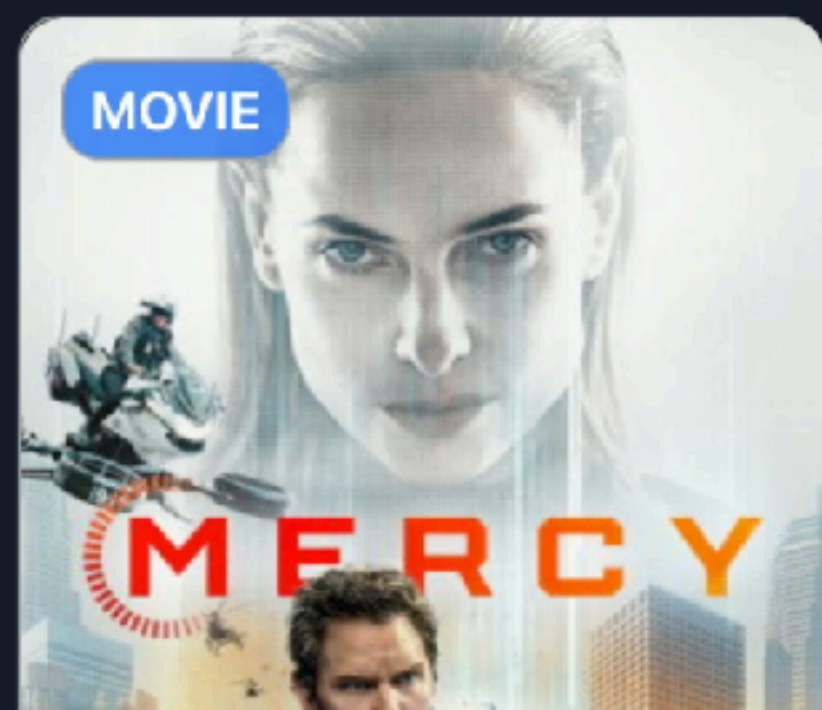
Search

Settings

Trending



Popular Movies



Demo

```
- npx create-expo-app MyTVProject -e with-router-tv
```


Thank you!



Feedback

Useful links

For your browsing pleasure

- **React-native-tvos repository**
<https://github.com/react-native-tvos/react-native-tvos>
- **Expo guide to building for tvOS**
<https://docs.expo.dev/guides/building-for-tv/>
- **React Native Directory (tvOS)**
<https://reactnative.directory/packages?tvos=true>
- **Article on focus management with TVFocusGuideView**
<https://medium.com/xite-engineering/revolutionizing-focus-management-in-tv-applications-with-react-native-10ba69bd90>
- **Apple Human Interface Guidelines**
<https://developer.apple.com/design/human-interface-guidelines/layout#tvOS>